



Group assignment

MODULE: SPEDS402: Data Structures and Algorithms with C++.

Learning Outcome: Implement Hash Table and handle Data Retrieval

Groups: 1,2,...,9

Group 1: Hash Function Design & Uniform Index Distribution

Core Focus: Principles of effective hash function design and mapping data to indices.

Task:

- Design three different mathematical hash functions in C++ for string inputs (e.g., division method, folding, and mid-square).
- Test each function with a dataset of 50 names to check how evenly they distribute indices.

Deliverables: A C++ script comparing the distribution patterns and a summary explaining which function minimized initial index overlaps.

Group 2: Collision Resolution using Linear Probing

Core Focus: Open addressing strategies and handling index distribution.

Task:

- Create a fixed-size integer hash table array in C++.
- Implement an insertion algorithm that handles collisions strictly using **Linear Probing**:

$$(index = (hash(key) + i) \text{ (mod size)})$$

- Write a lookup function to retrieve elements under this collision strategy.

Deliverables: Source code demonstrating successful resolution of at least 5 colliding keys, plus a demonstration of the primary clustering effect.

Group 3: Advanced Open Addressing (Quadratic Probing & Double Hashing)

Core Focus: Minimizing clustering issues in open addressing.

Task:

- Build a C++ hash table that resolves collisions using **Quadratic Probing** (i^2) and **Double Hashing** ($i \times hash_2(key)$).
- Run a step-by-step trace showing where items are stored when a collision happens.

Deliverables: Functional code containing both methods and a brief comparison chart highlighting how double hashing improves index distribution.

Group 4: Collision Resolution via Separate Chaining

Core Focus: Implementing dynamic linked lists inside a hash table array.

Task:

- Implement a hash table where each array index points to a singly linked list (Chaining technique).
- Code structural features to handle node insertion, key deletion, and sequential traversal within individual buckets.

Deliverables: Code demonstrating infinite collision capacity handling without changing the baseline array boundary.

Group 5: Memory Allocation & Load Factor Optimization

Core Focus: Dynamic memory allocation standards and performance scaling.

Task:

- Build a C++ hash table that tracks its own **Load Factor**

$$\left(\alpha = \frac{\text{items}}{\text{table size}}\right).$$

- Program an automatic "Rehashing" mechanism that doubles the array size and re-maps existing components once $(\alpha > 0)$.

Deliverables: Executable script showing the memory allocation update logs during large dataset insertion tests.

Group 6: Real-World Application – Dictionary & Spell-Checker

Core Focus: Adapting data retrieval to a specific text problem context.

Task:

- Create a prototype spell-checker using a hash table in C++.
- Load a mini-dictionary file into the table and accept text input to instantly verify if words are spelled correctly.

Deliverables: Interactive console application displaying instant $O(1)$ vocabulary lookup verification.

Group 7: Real-World Application – Frequency Counter Engine

Core Focus: Associative data storage, key-value handling, and retrieval mechanics.

Task:

- Build a character or word frequency counter using custom hash-mapping mechanics.
- Analyze an inputted paragraphs file, using words as unique keys and their occurrence totals as values.

Deliverables: Program output sorting or listing unique items paired directly with their total calculated densities.

Group 8: Performance Benchmark – Open Addressing vs. Chaining

Core Focus: Comparative analytical evaluation of collision resolution performance.

Task:

- Set up two identically sized hash tables: one using linear probing and one using separate chaining.

- Insert 1,000 random items to track lookup time, memory footprints, and retrieval failure rates under heavy collision strain.

Deliverables: Benchmark log tables showing how time complexity scales up for both setups as load capacity peaks.

Group 9: Secure Data Management – Student Record System

Core Focus: Data integrity preservation, collision tracking, and data storage design.

Task:

- Create a secure console registry system mapping unique Student Registration Numbers to comprehensive profile objects.
- Program clean functions for addition, immediate record modification, secure item deletion, and error-trapped key retrieval.

Deliverables: A fully CRUD-compliant database prototype built exclusively on top of a naked C++ hash structure.